

模型上下文协议导论

Introduction to Model Context Protocol

第三单元：客户端、资源与提示词

Module 3: Client, Resources, and Prompts

授课教师：

由 mooc2handout 自动生成

2026 年 7 月 30 日

本讲义由课程字幕、补充材料与可选的 AI 推断关键帧自动生成。

目录

本单元知识地图	3
0. 概要	3
1. 实现 MCP 客户端	3
2. 定义资源	3
3. 访问资源	4
4. 定义提示词	4
5. 在客户端使用提示词	5
6. 本单元最该记住的五句话	5
7. 练习题	5
8. 参考答案	6

本单元知识地图

主题	核心问题
Client session	它为什么不仅仅是一个请求对象？
资源注入	为什么 @ 能改善上下文输入？
Prompt 调用	list / get prompt 分别承担什么职责？

0. 概要

这一单元把 server 侧能力接回客户端，完成 resource 和 prompt 的读取、注入与交互。你会看到 MCP 不只是“把模型接到工具”，更是把应用里的上下文流转做成可组合的接口。

- client session 管理连接与清理。
- resources 适合做应用控制的数据注入。
- prompts 适合做用户控制的可复用模板。

1. 实现 MCP 客户端

回看：实现 MCP 客户端

本讲重点：把 server 侧能力接回 client，完成真正的往返调用。

这一讲开始实现 `client.py` 里的 MCP client 类。它会把 client session 包起来，让应用可以稳定地和 server 建立连接、发消息、收结果。

讲者特别提醒：这个类还负责收尾清理，所以它不是单纯的“发请求对象”，而是应用和外部 server 之间的生命周期管理者。

- client session 是真实连接的核心。
- client 代码比 server 更像应用基础设施的一部分。
- 连接管理和资源清理不能省。

自动摘要根据字幕自动扩写，这一讲再补充几条最值得记住的线索。

- **形式定义：**这一讲梳理 AND / OR / NOT / IF...THEN 等连接词的真值条件，并提醒不要把日常语言的直觉直接搬进数学里。

2. 定义资源

回看：定义资源

本讲重点：resources 让应用把上下文数据主动送给模型。

这一讲增加了一个很典型的交互：用户输入 @ 时，界面显示可引用的文档；用户选中文档后，应用会把该文档内容自动插入到发给 Claude 的 prompt 里。

这说明 resources 更像“应用控制的数据注入层”：它不是等模型自己去调用 tool，而是由应用先把合适的上下文准备好，再送进模型。

- 资源配合 exttt@ 自动补全，体验会更自然。
- 资源的内容会被插入 prompt，而不是让模型自己猜。
- 这是 app-controlled 的典型例子。

自动摘要根据字幕自动扩写，这一讲再补充几条最值得记住的线索。

- **形式定义**：这一讲梳理 AND / OR / NOT / IF...THEN 等连接词的真值条件，并提醒不要把日常语言的直觉直接搬进数学里。

3. 访问资源

回看：[访问资源](#)

本讲重点：client 需要主动向 server 取回资源内容。

在定义完资源之后，client 侧就要补上读取逻辑：给定一个 resource URI，发出请求、拿到响应，并根据 MIME type 把数据解析成应用能继续使用的形式。

这一步的意义在于把“资源”真正变成可消费的上下文，而不是只停留在 server 的名词定义上。

- client 需要知道如何按 URI 拉取资源。
- 不同 MIME type 需要不同的解析方式。
- 资源要最终变成 prompt 的一部分，才算真正用起来。

自动摘要根据字幕自动扩写，这一讲再补充几条最值得记住的线索。

- **定义**：这一讲补出一个后面会反复使用的定义，并说明它为什么重要。
- **形式定义**：这一讲梳理 AND / OR / NOT / IF...THEN 等连接词的真值条件，并提醒不要把日常语言的直觉直接搬进数学里。

4. 定义提示词

回看：[定义提示词](#)

本讲重点：prompts 让用户用 slash command 触发可复用模板。

这一讲引入 / 开头的命令，例如 `format`。用户一旦选择某个命令，应用就会把对应 prompt 当作一个可复用的操作模板来执行。

和 resources 不同，prompts 更强调“用户主动选择要执行什么”，因此它更接近交互式 UI 里的命令面板或菜单。

- slash command 是 prompts 的自然入口。
- prompts 适合做模板化、可复用的动作。
- 它更像用户控制的操作，而不是模型自动触发的动作。

自动摘要根据字幕自动扩写，这一讲再补充几条最值得记住的线索。

- **形式定义：**这一讲梳理 AND / OR / NOT / IF...THEN 等连接词的真值条件，并提醒不要把日常语言的直觉直接搬进数学里。

5. 在客户端使用提示词

回看：在客户端使用提示词

本讲重点：把 prompt 的发现和调用接到 client 里。

最后一个任务是实现 `list_prompts` 和 `get_prompt`：前者列出 server 提供的 prompt，后者把 prompt 名称和参数传进去，再把返回的 `messages` 交给 Claude。

这样一来，client 就不只是能拉资源，也能把模板化提示词真正接到应用交互里。用户在终端里看到的 slash command，背后其实就是这套消息流。

- `list_prompts` 负责发现 server 提供了哪些 prompt。
- `get_prompt` 负责把参数插值进 prompt。
- 返回的 `messages` 可以直接喂给 Claude 继续对话。

自动摘要根据字幕自动扩写，这一讲再补充几条最值得记住的线索。

- **形式定义：**这一讲梳理 AND / OR / NOT / IF...THEN 等连接词的真值条件，并提醒不要把日常语言的直觉直接搬进数学里。

6. 本单元最该记住的五句话

1. client session 负责管理与 server 的真实连接。
2. resources 负责把上下文数据插入 prompt。
3. prompts 让用户通过 slash command 触发模板。
4. client 侧要补齐读取资源与调用 prompt 的能力。
5. MCP 的重点之一是把上下文流转拆成清晰的接口。

7. 练习题

A. 选择题（5 题）

1. MCP client 中的 client session 主要负责什么？
A. 保存 UI 主题 B. 管理与 server 的连接和清理 C. 生成 prompt 模板 D. 存储文档正文
2. resources 的设计目标是什么？
A. 让用户输入 @ 时自动引用文档并注入上下文 B. 让模型训练参数 C. 让 server 自己写日志 D. 让 UI 更花哨

3. @ 触发的功能是什么？

- A. 自动删除文件 B. 显示可引用文档列表并插入内容 C. 创建 prompt D. 启动 Inspector

4. prompts 更像什么？

- A. 用户控制的 slash commands B. 随机种子 C. 数据库 schema D. 网络协议

5. get_prompt 返回的通常是什么？

- A. 文件路径 B. 一段 HTML C. 要直接喂给 Claude 的 messages D. 图片数组

B. 判断题 (3 题)

1. resources 的内容是由应用主动注入到 prompt 中的。()

2. prompts 更接近用户主动触发的操作。()

3. get_prompt 的返回值通常会直接进入后续对话流程。()

C. 简答题 (2 题)

1. 为什么文档资源适合通过 @ 这种交互方式引入？

2. 说明 resources 和 prompts 的区别。

8. 参考答案

选择题答案

1.B 2.A 3.B 4.A 5.C

判断题答案

1. 对 2. 对 3. 对

简答题要点

- 因为它可以把“引用文档”变成很自然的输入动作，随后由应用自动补足上下文。
- resources 由应用决定何时取用并注入数据；prompts 由用户决定何时触发一个模板化操作。